PreLAB: Timer Interrupt & PWM

Author: Won Bitna

Student ID: 22400462

Date: 2026-10-30

Part 1. Timer Interrupt

Exercise: TIMER Library (`ecTIM2.c`)

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Won Bitna
Created          : 05-03-2021
Modified         : 10-29-2026
Language/ver     : C++ in VS Code

Description      : Timer library
/-----------------------------------------------------------------*/

#include "ecTIM2.h"

////////////////////////////////////////////////////////////////
/*                      EXERCISE                              */
////////////////////////////////////////////////////////////////
/*
// Exercises are in these functions.
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);
*/




////////////////////////////////////////////
/* Timer Update Interrupt Configuration     */
////////////////////////////////////////////


void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Initialize Timer
    TIM_init(TIMx,msec);

// 2. Enable Update Interrupt
    TIM_UI_enable(TIMx);

// 3. NVIC Setting
    uint32_t IRQn_reg = get_TIM_IRQn(TIMx);
    NVIC_EnableIRQ(IRQn_reg);
    NVIC_SetPriority(IRQn_reg,2);
```

```
}

void TIM_UI_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Initialize Timer
    TIM_init_us(TIMx,usec);

// 2. Enable Update Interrupt
    TIM_UI_enable(TIMx);

// 3. NVIC Setting
    uint32_t IRQn_reg = get_TIM_IRQn(TIMx);
    NVIC_EnableIRQ(IRQn_reg);
    NVIC_SetPriority(IRQn_reg,2);
}


// Enable Timer Update Interrupt
void TIM_UI_enable(TIM_TypeDef* TIMx){
    TIMx->SR = ~TIM_SR_UIF;
    TIMx->DIER |= TIM_DIER_UIE;
}

// Disable Timer Update Interrupt
void TIM_UI_disable(TIM_TypeDef* TIMx){
    TIMx->DIER &= ~(TIM_DIER_UIE);
}

uint32_t is_UIF(TIM_TypeDef *TIMx){
    return TIMx->SR & TIM_SR_UIF;
}
void clear_UIF(TIM_TypeDef *TIMx){
    TIMx->SR &= ~TIM_SR_UIF;
}




////////////////////////////////////////////////////////////////
/* Timer Counter Initialization                               */
////////////////////////////////////////////////////////////////


// Timer Counter Initialization
// Default : f_cnt =  10kHz, msec= 1 to 6,553 msec
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period(TIMx, msec);

// 3. COUNTER Direction
    // Up-Counter
    TIMx->CR1 &= ~TIM_CR1_DIR;  // ~(0x1UL<<4);

    // Down-Counter
    //TIMx->CR1 |= TIM_CR1_DIR;
```

```
// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Timer Counter Initialization
// Default : f_cnt =  1 MHz, usec= 1 to 65,530 usec
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);
// 2. Set CNT period
    TIM_period_us(TIMx, usec);

// 3. CNT Direction
    TIMx->CR1 &= ~TIM_CR1_DIR;      // up-counter
// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Limits    : 16-bit ARR -> f_cnt =  10kHz, msec= 1 to 6,553 msec (ARR <
0xFFFF)
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz
(HSI)

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  10000UL;                    // 16-bit CNT: f_cnt=10kHz
(default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 10 kHz
(PSC=8400)
    TIMx->PSC = PSCval - 1;

    // Reload Value (ARR) for msec unit
    uint32_t ARRval = (f_cnt / 1000UL) * msec;      // Reload values needed for
msec
    TIMx->ARR = ARRval - 1;
}


// Limits    : 16-bit ARR -> f_cnt =  1 MHz, usec= 1 to 65,530 usec (ARR <
0xFFFF)
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz
(HSI)
    uint32_t ARRval=0;

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  1000000UL;                  // 16-bit CNT: f_cnt=1MHz
(default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 1 MHz (PSC=84)
    TIMx->PSC = PSCval - 1;
```

```
    //uReload Value (ARR) for msec unit
    ARRval = (f_cnt / 1000000UL) * usec;
    TIMx->ARR = ARRval - 1;
}



void TIM_clock_enable(TIM_TypeDef* TIMx){
    if(TIMx ==TIM1) RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;
    else if(TIMx ==TIM2) RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    else if(TIMx ==TIM3) RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    else if(TIMx ==TIM4) RCC->APB1ENR |= RCC_APB1ENR_TIM4EN;
    else if(TIMx ==TIM5) RCC->APB1ENR |= RCC_APB1ENR_TIM5EN;
    else if(TIMx ==TIM9) RCC->APB2ENR |= RCC_APB2ENR_TIM9EN;
    else if(TIMx ==TIM10) RCC->APB2ENR |= RCC_APB2ENR_TIM10EN;
    else if(TIMx ==TIM11) RCC->APB2ENR |= RCC_APB2ENR_TIM11EN;
}



uint32_t get_TIM_IRQn(TIM_TypeDef* TIMx){
    uint32_t IRQn_reg = 0x00000000;
    if(TIMx == TIM1)       IRQn_reg = TIM1_UP_TIM10_IRQn;
    else if(TIMx == TIM2)  IRQn_reg = TIM2_IRQn;
    else if(TIMx == TIM3)  IRQn_reg = TIM3_IRQn;
    else if(TIMx == TIM4)  IRQn_reg = TIM4_IRQn;
    else if(TIMx == TIM5)  IRQn_reg = TIM5_IRQn;
    else if(TIMx == TIM9)  IRQn_reg = TIM1_BRK_TIM9_IRQn;
    else if(TIMx == TIM10) IRQn_reg = TIM1_UP_TIM10_IRQn;
    else if(TIMx == TIM11) IRQn_reg = TIM1_TRG_COM_TIM11_IRQn;
    return IRQn_reg;
}
```

Part 2. PWM

Exercise: PWM Library (`ecPWM2.c`)

The following functions are completed from the exercise section of `ecPWM2.c`.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Won Bitna
Created          : 05-03-2021
Modified         : 10-29-2026
Language/ver     : C++ in VS Code

Description      : PWM library
*          PWM library - exercise version.
*          in ecPWM2_student.c .  This header needs no editing.

/----------------------------------------------------------------*/

#include "ecPWM2.h"
```

```
////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////


// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec){
    // 0. Match TIMx from given Port_y and Pin_x
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

    // 1. Initialize GPIO port and pin as AF
    GPIO_init(pinName, AF);

    // 2. Configure GPIO AFR by Pin x num.
    //  AFR[0] holds pin 0~7, AFR[1] holds pin 8~15 : 4 bits per pin
    uint32_t afrIdx   = pin >> 3;               // same as (pin/8)
    uint32_t afrShift = 4 * (pin % 8);
    uint32_t afNum    = 0;

    //  AF1 for TIM1,TIM2     AF2 for TIM3,TIM4
    if((TIMx == TIM1)||(TIMx == TIM2))      afNum = 1;  // AF1
    else if(TIMx== TIM3 || TIMx == TIM4)    afNum = 2;  // AF2
    // You can add more  for TIM5 to TIM11, if necessary!


    // Set AFR register for the pin
    port->AFR[afrIdx] &= ~(0xFUL << afrShift);  // 4 bits per pin
    port->AFR[afrIdx] |= (afNum <<afrShift);


    // 3. Initialize Timer : select CLK_CNT according to PWM period
    //   TIM2/TIM5          (32-bit ARR) : 1 < usec < 2^32 usec     ->
TIM_init_us()
    //   TIM1/TIM3/TIM4     (16-bit ARR) : 1 < usec <=65,536 usec   ->
TIM_init_us()
    //                                     65 < msec <65,536 msec  ->
TIM_init()
    if((TIMx == TIM2) || (TIMx == TIM5) || (usec <= 65536)){
        TIM_init_us(TIMx, usec);
    }
    else {
        uint32_t msec = usec / 1000;
        if(msec > 6553) msec = 6553;            // 16-bit ARR has limit of
6.5sec
        TIM_init(TIMx, msec);                   // 65 < msec <65,536 msec   ->
TIM_init()
    }
    // 4. Configure Timer Output mode as PWM
    // Default: PWM Mode 1, active high, duty ratio = 50%
    PWM_mode_config(TIMx, chN, EC_PWM_MODE1);
```

```
    // 5. Enable Timer Counter
    // For TIM1 ONLY
    if(TIMx == TIM1) TIMx->BDTR |= TIM_BDTR_MOE;                    // Main
output enable (MOE): 0 = Disable, 1 = Enable
    // All Timer: First, Clear the counter and Clear UI Flag
    TIMx->EGR |= TIM_EGR_UG;
    TIMx->SR   = ~TIM_SR_UIF;
    // Enable timers
    TIMx->CR1  |= TIM_CR1_CEN;
}


// Duty Ratio:  from 0.0 to 1.0
void PWM_duty(PinName_t pinName, float duty){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Check the duty ratio range from 0.0 to 1.0
    if (duty < 0.0f)    duty = 0.0f;
    if (duty > 1.0f)    duty = 1.0f;

// 2. Duty ratio -> CCR counts.  Assume PWM mode 1 :
    uint32_t CCvalue = (TIMx->ARR + 1) * duty;

    if(chN == 1)      { TIMx->CCR1 = CCvalue; } //set channel
    else if(chN == 2) { TIMx->CCR2 = CCvalue; }
    else if(chN == 3) { TIMx->CCR3 = CCvalue; }
    else if(chN == 4) { TIMx->CCR4 = CCvalue; }
}




////////////////////////////////////////////////
/*      GIVEN: PWM Initialization               */
////////////////////////////////////////////////

//  Low Frequency PWM initialization in msec. (below 1kHz)
//  Default:        50% duty ratio. PWM_mode1, Period in msec.
//  TIM2/TIM5       (32-bit ARR) : msec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) : msec 1 ~ 6,553
void PWM_init(PinName_t pinName, uint32_t msec){
    if(msec == 0) msec = 1;     // minimum 1 msec
    PWM_init_config(pinName, msec * 1000);
}

//  High-Frequency PWM initialization in usec (1kHz to 100 kHz)
//  Default:        50% duty ratio.  PWM_mode1, Period in usec.
//  TIM2/TIM5       (32-bit ARR) :  usec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) :  usec 1 ~ 6,553
void PWM_init_us(PinName_t pinName, uint32_t usec){
    if(usec < 10) usec = 10;  // Minimum 10us (100kHz)
    PWM_init_config(pinName, usec);
```

```
}


////////////////////////////////////////////////
/*      GIVEN: DUTY RATIO SETUP                */
////////////////////////////////////////////////



// Set Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

    float duty=0.0f;
    uint32_t f_cnt =0;


// 1. Counter clock : CK_CNT = EC_SYSCLK / (PSC+1)
    f_cnt = (uint32_t)EC_SYSCLK / (TIMx->PSC + 1);      // [Hz]

// 2. Convert Pulse_width [msec] -> duty ratio (0~1.0)
    uint32_t counts = (uint32_t)(((uint64_t)f_cnt * pulse_width_ms) / 1000UL);
    duty = (float)counts / (float)(TIMx->ARR + 1);

// 3. Set duty ratio
    PWM_duty(pinName, duty);
}


// Set Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, uint32_t pulse_width_us){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

    float duty=0.0f;

// 1. Counter clock : CK_CNT = EC_SYSCLK / (PSC+1)
    uint32_t f_cnt = (uint32_t)EC_SYSCLK / (TIMx->PSC + 1);     // [Hz]

// 2. Convert Pulse_width [usec] -> duty ratio (0~1.0)
    // Exercise for Students
    uint32_t counts = (uint32_t)(((uint64_t)f_cnt * pulse_width_us) /
1000000UL);
    duty = (float)counts / (float)(TIMx->ARR + 1);

// 3. Set duty ratio
    PWM_duty(pinName, duty);
```

```
}


////////////////////////////////////////////////////////////////
/* Internal Helper Functions - GIVEN. DO NOT MODIFY           */
////////////////////////////////////////////////////////////////

// Internal Helper function for PWM output configuration
void PWM_mode_config(TIM_TypeDef *TIMx, int chN, uint32_t PWMmode){

    int mode=0x6; // default value for PWM mode 1
    if (PWMmode == EC_PWM_MODE2)    mode=0x7;

    uint32_t ccVal = TIMx->ARR/2;  // default value  CC=ARR/2
    if(chN == 1){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC1M;                     // Clear ouput
compare mode bits for channel 1
        TIMx->CCMR1 |= ((mode << TIM_CCMR1_OC1M_Pos) & TIM_CCMR1_OC1M);   //
OCxM = 110 (PWM1) / 111 (PWM2) on ch1
        TIMx->CCMR1 |= TIM_CCMR1_OC1PE;                     // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR1  = ccVal;                                // Output Compare
Register for channel 1 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC1P;                       // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC1E;                       // Enable output for
ch1
    }
    else if(chN == 2){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC2M;                     // Clear ouput
compare mode bits for channel 2
        TIMx->CCMR1 |= ((mode << TIM_CCMR1_OC2M_Pos) & TIM_CCMR1_OC2M);   //
OCxM = 110 (PWM1) / 111 (PWM2) on ch2
        TIMx->CCMR1 |= TIM_CCMR1_OC2PE;                     // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR2  = ccVal;                                // Output Compare
Register for channel 2 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC2P;                       // select output
polarity: active high
        TIMx->CCER |= TIM_CCER_CC2E;                        // Enable output for
ch2
    }
    else if(chN == 3){
        TIMx->CCMR2 &= ~TIM_CCMR2_OC3M;                     // Clear ouput
compare mode bits for channel 3
        TIMx->CCMR2 |= ((mode << TIM_CCMR2_OC3M_Pos) & TIM_CCMR2_OC3M);   //
OCxM = 110 (PWM1) / 111 (PWM2) on ch3
        TIMx->CCMR2 |= TIM_CCMR2_OC3PE;                     // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR3  = ccVal;                                // Output Compare
Register for channel 3 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC3P;                       // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC3E;                       // Enable output for
ch3
    }
    else if(chN == 4){
```

```
        TIMx->CCMR2 &= ~TIM_CCMR2_OC4M;                     // Clear ouput
compare mode bits for channel 4
        TIMx->CCMR2 |= ((mode << TIM_CCMR2_OC4M_Pos) & TIM_CCMR2_OC4M);   //
OCxM = 110 (PWM1) / 111 (PWM2) on ch4
        TIMx->CCMR2 |= TIM_CCMR2_OC4PE;                     // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR4  = ccVal;                                // Output Compare
Register for channel 4 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC4P;                       // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC4E;                       // Enable output for
ch4
    }
}


// Internal Helper function for PWM Pin Matching
void PWM_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN)
{
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);

    // Only  TIM1 to TIM4 are defined.
    // TIM5, 9 to 11 needs to be added, if necessary
  if(port == GPIOA) {
      switch(pin){
         case 0 : *TIMx = TIM2; *chN    = 1; break;
         case 1 : *TIMx = TIM2; *chN = 2; break;
         case 5 : *TIMx = TIM2; *chN = 1; break;
         case 6 : *TIMx = TIM3; *chN = 1; break;
         //case 7: TIMx = TIM1; *chN = 1N; break;
         case 8 : *TIMx = TIM1; *chN = 1; break;
         case 9 : *TIMx = TIM1; *chN = 2; break;
         case 10: *TIMx = TIM1; *chN = 3; break;
         case 15: *TIMx = TIM2; *chN = 1; break;
         default: break;
      }
   }
   else if(port == GPIOB) {
      switch(pin){
         //case 0: TIMx = TIM1; *chN = 2N; break;
         //case 1: TIMx = TIM1; *chN = 3N; break;
         case 3 : *TIMx = TIM2; *chN = 2; break;
         case 4 : *TIMx = TIM3; *chN = 1; break;
         case 5 : *TIMx = TIM3; *chN = 2; break;
         case 6 : *TIMx = TIM4; *chN = 1; break;
         case 7 : *TIMx = TIM4; *chN = 2; break;
         case 8 : *TIMx = TIM4; *chN = 3; break;
         case 9 : *TIMx = TIM4; *chN = 4; break;
         case 10: *TIMx = TIM2; *chN = 3; break;
         default: break;
      }
   }
   else if(port == GPIOC) {
      switch(pin){
         case 6 : *TIMx = TIM3; *chN = 1; break;
         case 7 : *TIMx = TIM3; *chN = 2; break;
```

```
          case 8 : *TIMx = TIM3; *chN = 3; break;
          case 9 : *TIMx = TIM3; *chN = 4; break;
          default: break;
       }
   }
    // TIM5, 9, 11 needs to be added, if used.
    // if necessary
}
```